LangChain & Mem0

Insurance AI Agent Bootcamp

Hands-On Lab Implementation Guide

Module 1	LangChain Agents – Claim Validation Agent
Module 2	Tool Calling – External API Simulation
Module 3	ReAct Agent – Loan Eligibility Reasoning
Module 4	LangChain Memory – Conversational Chatbot
Module 5	Mem0 + Vector DB – Personalised Sessions

AWS EC2 Ubuntu 22.04	**OpenAI API GPT-4o-mini**	**LangChain 0.3+**	**Mem0 Vector DB**	**Code-Server Browser IDE**

Table of Contents

Prerequisites & Environment Setup

All hands-on labs in this guide run on an **AWS EC2 Ubuntu 22.04** instance with **Code-Server** (VS Code in the browser) pre-installed. Each participant has been provided an **OpenAI API Key**. Follow every step below before attempting any module.

1 · System Requirements

Component	Requirement
EC2 Instance Type	t3.medium or t3.large (minimum 2 vCPU, 4 GB RAM)
OS	Ubuntu 22.04 LTS
Python	3.10 or 3.11 (pre-installed)
Code-Server	Accessible on port 8080 via browser
OpenAI API Key	Provided by trainer – keep it confidential
Internet Access	Required for pip installs and OpenAI API calls

2 · Open a Terminal in Code-Server

Launch Code-Server in your browser. Navigate to **Terminal** → **New Terminal** from the top menu. All commands below are executed in this terminal.

3 · Create a Dedicated Project Directory

```
# Create and enter the project directory mkdir -p ~/insurance-ai-lab && cd ~/insurance-ai-lab # Create a Python virtual environment python3 -m venv venv # Activate the virtual environment source venv/bin/activate # Confirm activation (prompt will show "(venv)") which python
```

4 · Install All Required Libraries

Install all dependencies in one step. This covers every module in the bootcamp.

```
pip install --upgrade pip # Core LangChain stack pip install langchain langchain-openai langchain-community langchain-core # Tool calling & agents pip install openai # Vector database for Module 5 pip install qdrant-client # Mem0 for persistent memory (Module 5) pip install mem0ai # Utilities pip install python-dotenv requests colorama
```

■ ***Note:*** *If you see any 'dependency conflict' warnings, they can be safely ignored for this lab. All core functionality will work correctly.*

5 · Configure the OpenAI API Key

Store the key in a **.env** file so it is loaded automatically in every module. Never hard-code keys in source files.

```
# Create the .env file (replace with your actual key) echo 'OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx' > .env # Verify cat .env
```

In every Python file you will load it with:

```
from dotenv import load_dotenv import os load_dotenv() # reads .env into environment variables api_key = os.getenv("OPENAI_API_KEY")
```

■ **Important: Never commit the .env file to Git or share its contents. Treat your API key like a password.**

6 · Verify Installation

```
python3 -c " import langchain, openai, mem0 print('LangChain:', langchain.__version__) print('OpenAI:', openai.__version__) print('Mem0: OK') print('All libraries loaded successfully!') "
```

Expected output:

```
LangChain: 0.3.x OpenAI: 1.x.x Mem0: OK All libraries loaded successfully!
```

Module 1 · Claim Validation Agent

Build a LangChain agent that validates insurance claims via tool calls

LangChain Agents OpenAI GPT-4o-mini Tool Binding BFSI Use-Case

■ Learning Objectives

✓ Understand the LangChain Agent execution loop (Thought → Action → Observation)

✓ Define and register custom Python functions as agent tools

✓ Connect an OpenAI LLM to the agent executor

✓ Execute multi-step claim validation with tool orchestration

✓ Interpret agent scratchpad traces for debugging

1.1 · Architecture Overview

A LangChain Agent wraps an LLM with the ability to use external tools. The agent reasons about which tool to call, invokes it, receives the result, and then decides the next step — repeating until it reaches a final answer. In this module we build a **Claim Validation Agent** for an insurance BFSI scenario that checks policy existence, verifies claim amounts, and detects fraud signals.

1.2 · Create the Project File

```
# In Code-Server terminal touch ~/insurance-ai-lab/module1_claim_agent.py # Open the file in Code-Server editor and add the code below
```

1.3 · Step-by-Step Implementation

Step 1 · Imports and Environment Setup

```
# module1_claim_agent.py from dotenv import load_dotenv import os from langchain_openai import ChatOpenAI from langchain.agents import AgentExecutor, create_tool_calling_agent from langchain_core.tools import tool from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder load_dotenv() # Load OPENAI_API_KEY from .env
```

Step 2 · Simulated Policy Database

We simulate a policy database using a Python dictionary. In production, this would connect to a real database or REST API.

```
# Simulated insurance policy database POLICY_DB = { "POL-1001": { "holder": "Rajesh Kumar", "type": "Health", "max_coverage": 500000, "active": True, "premium_paid": True }, "POL-1002": { "holder": "Priya Sharma", "type": "Motor", "max_coverage": 200000, "active": True, "premium_paid": False # lapsed policy }, "POL-1003": { "holder": "Amit Patel", "type": "Home", "max_coverage": 1000000, "active": False, # cancelled policy "premium_paid": True }, } # Simulated fraud signals database FRAUD_FLAGS = { "CLM-9999": "Multiple claims within 30 days", "CLM-8888": "Amount exceeds historical average by 300%", }
```

Step 3 · Define Agent Tools using @tool Decorator

The **@tool** decorator converts any Python function into a LangChain tool. The docstring becomes the tool description that the LLM uses to decide when to call it.

```
@tool def verify_policy(policy_id: str) -> str: """ Verifies if an insurance policy exists and is active. Returns policy details including coverage amount and status. Use this tool first before processing any claim. Args: policy_id: The unique policy identifier (e.g., POL-1001) """ policy = POLICY_DB.get(policy_id) if not policy: return f"ERROR: Policy {policy_id} not found in database." if not policy["active"]: return (f"REJECTED: Policy {policy_id} for {policy['holder']} " f"is CANCELLED and not eligible for claims.") if not policy["premium_paid"]: return (f"REJECTED: Policy {policy_id} for {policy['holder']} " f"has LAPSED due to non-payment. Claim cannot be processed.") return (f"VALID: Policy {policy_id} - Holder: {policy['holder']}, " f"Type: {policy['type']}, " f"Max Coverage: Rs.{policy['max_coverage']:,}, " f"Status: Active") @tool def validate_claim_amount(policy_id: str, claim_amount: float) -> str: """ Validates if the claimed amount is within policy coverage limits. Must be called after verify_policy confirms the policy is active. Args: policy_id: The unique policy identifier claim_amount: The amount being claimed in Indian Rupees """ policy = POLICY_DB.get(policy_id) if not policy: return f"ERROR: Policy {policy_id} not found." max_coverage = policy["max_coverage"] if claim_amount <= 0: return "INVALID: Claim amount must be greater than zero." if claim_amount > max_coverage: return (f"REJECTED: Claim amount Rs.{claim_amount:,.0f} exceeds " f"maximum coverage of Rs.{max_coverage:,} for policy {policy_id}.") percentage = (claim_amount / max_coverage) * 100 return (f"APPROVED: Claim amount Rs.{claim_amount:,.0f} is within " f"coverage limit. ({percentage:.1f}% of maximum coverage used)") @tool def check_fraud_signals(claim_id: str) -> str: """ Checks if a claim has any fraud indicators or suspicious patterns. Always call this tool as part of the claim validation process. Args: claim_id: The unique claim identifier (e.g., CLM-1001) """ flag = FRAUD_FLAGS.get(claim_id) if flag: return (f"FRAUD ALERT: Claim {claim_id} has been flagged. " f"Reason: {flag}. Escalate to fraud investigation team.") return (f"CLEAR: No fraud signals detected for claim {claim_id}. " f"Claim can proceed to approval workflow.") @tool def calculate_settlement(claim_amount: float, policy_type: str) -> str: """ Calculates the final settlement amount after applying deductibles and co-payment rules based on policy type. Args: claim_amount: The approved claim amount in Indian Rupees policy_type: Type of insurance (Health, Motor, Home) """ deductibles = {"Health": 0.10, "Motor": 0.15, "Home": 0.05} deductible_rate = deductibles.get(policy_type, 0.10) deductible_amount = claim_amount * deductible_rate settlement = claim_amount - deductible_amount return (f"Settlement Calculation for {policy_type} policy:\n" f" Claimed Amount: Rs.{claim_amount:,.0f}\n" f" Deductible ({deductible_rate*100:.0f}%): Rs.{deductible_amount:,.0f}\n" f" Final Settlement: Rs.{settlement:,.0f}")
```

Step 4 · Initialise the LLM and Build the Agent

```
# Initialise GPT-4o-mini llm = ChatOpenAI( model="gpt-4o-mini", temperature=0, # deterministic for claim processing api_key=os.getenv("OPENAI_API_KEY") ) # Define the prompt template for the agent prompt = ChatPromptTemplate.from_messages([ ("system", """You are an expert Insurance Claim Processing Agent for an Indian insurance company.
```

```
Your role is to validate insurance claims systematically and thoroughly. VALIDATION
PROTOCOL: 1. Always start by verifying the policy exists and is active 2. Validate the
claim amount against coverage limits 3. Check for any fraud signals 4. If all checks
pass, calculate the final settlement amount 5. Provide a clear, professional final
recommendation Be precise, methodical, and always explain your reasoning."""),
MessagesPlaceholder(variable_name="chat_history", optional=True), ("human", "{input}"),
MessagesPlaceholder(variable_name="agent_scratchpad"), ]) # Bundle all tools tools =
[verify_policy, validate_claim_amount, check_fraud_signals, calculate_settlement] #
Create the agent agent = create_tool_calling_agent(llm, tools, prompt) # Wrap agent with
executor (handles the ReAct loop) agent_executor = AgentExecutor( agent=agent,
tools=tools, verbose=True, # prints reasoning steps to terminal max_iterations=10,
handle_parsing_errors=True, return_intermediate_steps=True )
```

Step 5 · Run Test Scenarios

```
def run_claim_validation(query: str) -> None: """Execute agent and print formatted
output.""" print("\n" + "="*60) print(f"CLAIM QUERY: {query}") print("="*60) result =
agent_executor.invoke({"input": query}) print("\n[FINAL DECISION]")
print(result["output"]) print("="*60) if __name__ == "__main__": # Test Case 1: Valid
claim within limits run_claim_validation( "Process claim CLM-1001 for policy POL-1001. "
"The customer is claiming Rs.75,000 for a medical emergency." ) # Test Case 2: Lapsed
policy run_claim_validation( "Validate claim CLM-1002 for policy POL-1002. " "Motor
accident claim for Rs.45,000." ) # Test Case 3: Fraud flag detected
run_claim_validation( "Process claim CLM-9999 for policy POL-1001. " "Claiming Rs.25,000
for outpatient treatment." ) # Test Case 4: Amount exceeds coverage
run_claim_validation( "Customer with policy POL-1001 (Health) is claiming " "Rs.800,000
for a surgical procedure. Validate this claim." )
```

1.4 · Running the Application

```
# In Code-Server terminal cd ~/insurance-ai-lab source venv/bin/activate python3
module1_claim_agent.py
```

You will see the agent's step-by-step reasoning (because verbose=True):

```
> Entering new AgentExecutor chain... Invoking: `verify_policy` with {'policy_id':
'POL-1001'} VALID: Policy POL-1001 - Holder: Rajesh Kumar, Type: Health... Invoking:
`validate_claim_amount` with {'policy_id': 'POL-1001', 'claim_amount': 75000} APPROVED:
Claim amount Rs.75,000 is within coverage limit... Invoking: `check_fraud_signals` with
{'claim_id': 'CLM-1001'} CLEAR: No fraud signals detected for claim CLM-1001...
Invoking: `calculate_settlement` with {'claim_amount': 75000.0, 'policy_type': 'Health'}
Settlement Calculation for Health policy: Claimed Amount: Rs.75,000 Deductible (10%):
Rs.7,500 Final Settlement: Rs.67,500 [FINAL DECISION] Claim CLM-1001 has been validated
successfully. Final settlement amount is Rs.67,500 after applying the 10% health policy
deductible. > Finished chain.
```

Module 2 · Tool Calling – External API Simulation

Add structured tool calling to your agent with simulated external APIs

Tool Calling Structured Output API Simulation Pydantic

■ **Learning Objectives**

✓ Define tools with Pydantic input schemas for type safety

✓ Simulate external REST API calls (credit bureau, hospital network, RTO)

✓ Handle tool call errors gracefully within the agent loop

✓ Use parallel tool calling for independent data lookups

✓ Log and inspect tool call payloads for debugging

2.1 · Architecture Overview

Module 2 extends the concept of agents by introducing **structured tool calling** using Pydantic models. OpenAI function calling requires a JSON schema for each tool — Pydantic auto-generates this schema, giving us type validation for free. We simulate three external APIs commonly used in Indian insurance BFSI workflows: a credit bureau API, a hospital network API, and an RTO (vehicle registration) API.

2.2 · Create the File

```
touch ~/insurance-ai-lab/module2_tool_calling.py
```

2.3 · Full Implementation

Step 1 · Imports

```
# module2_tool_calling.py from dotenv import load_dotenv import os, json, random, time
from typing import Optional from langchain_openai import ChatOpenAI from
langchain_core.tools import tool from langchain.agents import AgentExecutor,
create_tool_calling_agent from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder load_dotenv()
```

Step 2 · Simulated External API Functions

```
# ■■ Simulated Credit Bureau API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def _call_credit_bureau_api(pan_number: str) -> dict: """Simulates an external Credit
Bureau REST API call.""" time.sleep(0.3) # simulate network latency credit_data = {
"ABCDE1234F": {"score": 780, "outstanding_loans": 2, "dpd": 0, "risk": "Low"},
"XYZAB9876K": {"score": 620, "outstanding_loans": 5, "dpd": 45, "risk": "High"},
"PQRST5678M": {"score": 720, "outstanding_loans": 1, "dpd": 0, "risk": "Medium"}, } data
= credit_data.get(pan_number.upper()) if not data: return {"error": f"PAN {pan_number}
not found in credit bureau"} return {"pan": pan_number, **data, "status": "success",
"bureau": "CIBIL Simulated"} # ■■ Simulated Hospital Network API
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ def _call_hospital_api(hospital_code:
str, patient_id: str) -> dict: """Simulates an insurance company's hospital network
```

```
API.""" time.sleep(0.2) hospitals = { "HOSP-001": {"name": "Apollo Hospitals",
"network": "Tier-1", "cashless": True, "city": "Hyderabad"}, "HOSP-002": {"name": "Care
Hospitals", "network": "Tier-2", "cashless": True, "city": "Pune"}, "HOSP-003": {"name":
"Local Clinic", "network": None, "cashless": False, "city": "Nagpur"}, } hosp =
hospitals.get(hospital_code) if not hosp: return {"error": f"Hospital {hospital_code}
not in network database"} return { "hospital": hosp, "patient_admitted": True,
"admission_date": "2024-03-15", "diagnosis_code": "J18.9", "estimated_cost":
random.randint(40000, 150000), "cashless_approved": hosp["cashless"] } # ■■ Simulated
RTO Vehicle Registration API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ def
_call_rto_api(vehicle_number: str) -> dict: """Simulates the RTO vehicle registration
database API.""" time.sleep(0.2) vehicles = { "TS09EF1234": {"owner": "Rajesh Kumar",
"make": "Maruti Suzuki", "model": "Swift", "year": 2021, "rc_valid": True,
"insurance_valid": True}, "MH12AB5678": {"owner": "Priya Sharma", "make": "Honda",
"model": "City", "year": 2019, "rc_valid": True, "insurance_valid": False}, } v =
vehicles.get(vehicle_number.upper()) if not v: return {"error": f"Vehicle
{vehicle_number} not found in RTO database"} return {"vehicle_number": vehicle_number,
**v, "status": "success"}
```

Step 3 · Wrap APIs as LangChain Tools

```
@tool def get_credit_score(pan_number: str) -> str: """ Fetches credit score and
financial risk profile from the CIBIL bureau. Essential for loan and high-value
insurance applications. Args: pan_number: PAN card number of the applicant (e.g.,
ABCDE1234F) """ result = _call_credit_bureau_api(pan_number) if "error" in result:
return f"API Error: {result['error']}" return (f"Credit Report [{result['bureau']}]:\n"
f" CIBIL Score: {result['score']}/900\n" f" Risk Category: {result['risk']}\n" f" Active
Loans: {result['outstanding_loans']}\n" f" Days Past Due: {result['dpd']} days") @tool
def verify_hospitalisation(hospital_code: str, patient_id: str) -> str: """ Verifies
patient hospitalisation details and cashless facility eligibility. Use for health
insurance claims involving hospitalisation. Args: hospital_code: Hospital network code
(e.g., HOSP-001) patient_id: Patient's unique identifier """ result =
_call_hospital_api(hospital_code, patient_id) if "error" in result: return f"Hospital
API Error: {result['error']}" h = result["hospital"] cashless_status = "APPROVED" if
result["cashless_approved"] else "NOT AVAILABLE" return (f"Hospitalisation Verified:\n"
f" Hospital: {h['name']} ({h['network'] or 'Out-of-Network'})\n" f" City: {h['city']}\n"
f" Cashless Facility: {cashless_status}\n" f" Admission Date:
{result['admission_date']}\n" f" Estimated Cost: Rs.{result['estimated_cost']:,}") @tool
def lookup_vehicle_registration(vehicle_number: str) -> str: """ Looks up vehicle
registration details from the RTO database. Required for motor insurance claim
validation. Args: vehicle_number: Vehicle registration number (e.g., TS09EF1234) """
result = _call_rto_api(vehicle_number) if "error" in result: return f"RTO API Error:
{result['error']}" status = "✓ VALID" if result["rc_valid"] else "✗ EXPIRED" ins_status
= "✓ VALID" if result["insurance_valid"] else "✗ EXPIRED" return (f"RTO Registration
Details:\n" f" Vehicle: {result['make']} {result['model']} ({result['year']})\n" f"
Registered Owner: {result['owner']}\n" f" RC Status: {status}\n" f" Current Insurance:
{ins_status}")
```

Step 4 · Agent Setup and Test Queries

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0,
api_key=os.getenv("OPENAI_API_KEY")) prompt = ChatPromptTemplate.from_messages([
("system", """You are an Insurance Processing Assistant with access to external data
APIs. Use the available tools to gather required information before making decisions.
Always call relevant tools before giving a final answer. For motor claims, check vehicle
registration. For health claims, verify hospitalisation. For any financial risk
assessment, check credit scores."""), ("human", "{input}"),
MessagesPlaceholder(variable_name="agent_scratchpad"), ]) tools = [get_credit_score,
```

```
verify_hospitalisation, lookup_vehicle_registration] agent =
create_tool_calling_agent(llm, tools, prompt) agent_executor =
AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=8) if __name__ ==
"__main__": # Test 1: Health insurance with hospitalisation print("\n--- TEST 1: Health
Claim ---") result = agent_executor.invoke({ "input": "Verify health claim for patient
PT-5001 admitted at HOSP-001. " "Also check credit risk for PAN ABCDE1234F. " "Summarise
all findings." }) print("RESULT:", result["output"]) # Test 2: Motor insurance claim
print("\n--- TEST 2: Motor Claim ---") result = agent_executor.invoke({ "input":
"Process motor claim for vehicle TS09EF1234. " "Check all documents and advise if claim
can proceed." }) print("RESULT:", result["output"])
```

2.4 · Running the Application

```
python3 module2_tool_calling.py
```

■ ***Note:*** *Notice how the agent calls multiple tools in sequence or in parallel depending on the query. The verbose output shows exactly which tool was called with which arguments.*

Module 3 · ReAct Agent – Loan Eligibility Reasoning

Implement the ReAct (Reason+Act) paradigm for multi-step decision making

ReAct Pattern Structured Reasoning Loan Assessment Chain-of-Thought

■ Learning Objectives

✓ Understand the ReAct (Reason → Act → Observe → Repeat) execution pattern

✓ Build a multi-factor loan eligibility agent for an NBFC/bank use case

✓ Implement tools for income verification, CIBIL check, and EMI calculation

✓ Force structured step-by-step reasoning using a custom system prompt

✓ Handle edge cases: insufficient data, borderline eligibility, co-applicants

3.1 · ReAct Architecture

ReAct (**Re**asoning + **Act**ing) is an agent paradigm where the model explicitly alternates between *Thought*, *Action*, and *Observation* steps. This creates an interpretable audit trail — critical in BFSI for regulatory compliance and loan adjudication documentation.

Step	What Happens	Example
Thought	LLM reasons about what to do next	"I need to check the applicant's income first"
Action	LLM calls a specific tool	→ verify_income(applicant_id='AP-101')
Observation	Tool returns a result	Income: Rs.75,000/month, Stable employment
Thought	LLM processes the result	"Income verified. Now I need CIBIL score."
Action	LLM calls next tool	→ check_cibil(pan='ABCDE1234F')
...	Continues until final answer	Final eligibility decision with reasoning

3.2 · Implementation

```
touch ~/insurance-ai-lab/module3_react_agent.py
```

Step 1 · Domain Tools for Loan Assessment

```
# module3_react_agent.py from dotenv import load_dotenv import os from langchain_openai import ChatOpenAI from langchain_core.tools import tool from langchain.agents import AgentExecutor, create_tool_calling_agent from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder load_dotenv() # Simulated applicant database APPLICANTS = { "AP-101": {"name": "Suresh Nair", "age": 35, "employment": "Salaried", "monthly_income": 75000, "years_employed": 8, "existing_emis": 8000, "pan": "ABCDE1234F"}, "AP-102": {"name": "Meera Joshi", "age": 28, "employment": "Self-Employed", "monthly_income": 120000, "years_employed": 5, "existing_emis": 25000, "pan": "PQRST5678M"}, "AP-103": {"name": "Ravi Krishnan", "age": 55, "employment":
```

```
"Salaried", "monthly_income": 45000, "years_employed": 30, "existing_emis": 15000,
"pan": "XYZAB9876K"}, } CIBIL_DB = { "ABCDE1234F": 780, "PQRST5678M": 720, "XYZAB9876K":
620 } @tool def get_applicant_profile(applicant_id: str) -> str: """ Fetches the loan
applicant's basic profile including income, employment type, age, and existing EMI
obligations. Always call this tool first to understand the applicant. Args:
applicant_id: Unique applicant ID (e.g., AP-101) """ a = APPLICANTS.get(applicant_id) if
not a: return f"ERROR: Applicant {applicant_id} not found." return (f"Applicant
Profile:\n" f" Name: {a['name']}, Age: {a['age']} years\n" f" Employment:
{a['employment']} ({a['years_employed']} years)\n" f" Monthly Income:
Rs.{a['monthly_income']:,}\n" f" Existing EMIs: Rs.{a['existing_emis']:,}/month\n" f"
PAN: {a['pan']}") @tool def check_cibil_score(pan_number: str) -> str: """ Fetches the
CIBIL credit score for a given PAN number. Scores >= 750 are excellent, 700-749 good,
650-699 fair, below 650 poor. Args: pan_number: PAN card number of the applicant """
score = CIBIL_DB.get(pan_number.upper()) if not score: return f"No CIBIL record found
for PAN {pan_number}" if score >= 750: category = "Excellent - Fast-track approval
eligible" elif score >= 700: category = "Good - Standard approval process" elif score >=
650: category = "Fair - Additional documents may be required" else: category = "Poor -
High risk; likely rejection or high interest rate" return f"CIBIL Score: {score}/900 -
{category}" @tool def calculate_loan_eligibility( monthly_income: float, existing_emis:
float, requested_amount: float, tenure_months: int, interest_rate: float = 10.5) -> str:
""" Calculates loan eligibility using the FOIR (Fixed Obligation to Income Ratio) method
and checks if the EMI fits within permissible limits. RBI guidelines allow maximum FOIR
of 50-55% for most banks. Args: monthly_income: Gross monthly income in rupees
existing_emis: Total existing monthly EMI obligations requested_amount: Loan amount
requested tenure_months: Requested loan tenure in months interest_rate: Annual interest
rate percentage (default 10.5%) """ # EMI Calculation using standard formula r =
(interest_rate / 100) / 12 # monthly rate new_emi = requested_amount * r *
(1+r)**tenure_months / ((1+r)**tenure_months - 1) total_obligations = existing_emis +
new_emi foir = (total_obligations / monthly_income) * 100 max_eligible_emi =
(monthly_income * 0.50) - existing_emis max_loan = max_eligible_emi *
((1+r)**tenure_months - 1) / (r * (1+r)**tenure_months) status = "ELIGIBLE" if foir <=
50 else "INELIGIBLE (FOIR exceeded)" return (f"Loan Eligibility Analysis:\n" f"
Requested Amount: Rs.{requested_amount:,.0f}\n" f" Calculated EMI:
Rs.{new_emi:,.0f}/month\n" f" Current FOIR: {foir:.1f}% (max allowed: 50%)\n" f" Status:
{status}\n" f" Maximum Eligible Loan: Rs.{max(0, max_loan):,.0f}") @tool def
assess_loan_risk( cibil_score: int, employment_type: str, years_employed: int, age: int)
-> str: """ Provides a holistic risk assessment combining multiple factors. Use this
after gathering all other information for a final risk rating. Args: cibil_score:
Applicant's CIBIL credit score employment_type: 'Salaried' or 'Self-Employed'
years_employed: Number of years in current employment/business age: Age of applicant in
years """ risk_score = 0 reasons = [] if cibil_score >= 750: risk_score += 30 elif
cibil_score >= 700: risk_score += 20; reasons.append("Moderate credit score") elif
cibil_score >= 650: risk_score += 10; reasons.append("Below-average credit score") else:
reasons.append("Poor credit score - high default risk") if employment_type ==
"Salaried": risk_score += 25 else: risk_score += 15; reasons.append("Self-employed
income less stable") if years_employed >= 5: risk_score += 20 elif years_employed >= 2:
risk_score += 10; reasons.append("Limited employment history") else:
reasons.append("Very short employment history") if 25 <= age <= 50: risk_score += 25
elif age < 25: risk_score += 10; reasons.append("Young applicant, shorter repayment
window") else: risk_score += 15; reasons.append("Age may limit loan tenure") if
risk_score >= 80: category = "LOW RISK - Preferential rate eligible" elif risk_score >=
60: category = "MEDIUM RISK - Standard terms" elif risk_score >= 40: category = "HIGH
RISK - Collateral or guarantor required" else: category = "VERY HIGH RISK - Recommend
rejection" reason_str = "; ".join(reasons) if reasons else "No negative factors
identified" return (f"Risk Assessment Score: {risk_score}/100\n" f" Category:
```

```
{category}\n" f" Key Concerns: {reason_str}")
```

Step 2 · ReAct Agent with Structured Reasoning Prompt

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0,
api_key=os.getenv("OPENAI_API_KEY")) REACT_SYSTEM_PROMPT = """You are a Loan Eligibility Assessment Officer at an Indian bank. You must evaluate loan applications using a strict ReAct reasoning process. MANDATORY ASSESSMENT SEQUENCE: Step 1 - THOUGHT: Identify what information you need Step 2 - ACTION: Call get_applicant_profile to get basic details Step 3 - THOUGHT: Note the income, age, employment, existing EMIs Step 4 - ACTION: Call check_cibil_score with the applicant's PAN Step 5 - THOUGHT: Evaluate the credit score implication Step 6 - ACTION: Call calculate_loan_eligibility with the financial details Step 7 - THOUGHT: Check if FOIR is within acceptable limits Step 8 - ACTION: Call assess_loan_risk for holistic risk rating Step 9 - THOUGHT: Synthesize all findings Step 10 - FINAL ANSWER: Provide structured recommendation with: - Approval/Rejection/Conditional Approval decision - Recommended loan amount (if different from requested) - Suggested interest rate based on risk - Required conditions (if any) - Brief reasoning (3-4 sentences) Always show your reasoning at each step. Be thorough and precise.""" prompt = ChatPromptTemplate.from_messages([ ("system", REACT_SYSTEM_PROMPT), ("human", "{input}"), MessagesPlaceholder(variable_name="agent_scratchpad"), ]) tools = [get_applicant_profile, check_cibil_score, calculate_loan_eligibility, assess_loan_risk] agent = create_tool_calling_agent(llm, tools, prompt) agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=15, handle_parsing_errors=True) if __name__ == "__main__": test_cases = [ ("AP-101", 1500000, 60, "Evaluate home loan for applicant AP-101: requesting Rs.15 lakh for 5 years."), ("AP-102", 5000000, 120, "Personal loan application AP-102: Rs.50 lakh for 10 years at business expansion."), ("AP-103", 800000, 24, "Vehicle loan for AP-103: Rs.8 lakh for 24 months. Process full assessment."), ] for aid, amt, tenure, query in test_cases: print(f"\n{'='*65}") print(f"CASE: {query}") print('='*65) result = agent_executor.invoke({"input": query}) print("\nFINAL RECOMMENDATION:", result["output"])
```

3.4 · Running the ReAct Agent

```
python3 module3_react_agent.py
```

■ ***Note:*** *The verbose output will show the full ReAct chain — Thought, Action, Observation, Thought, Action... — for each loan application. This audit trail is what makes ReAct valuable in regulated BFSI environments.*

Module 4 · Conversational Chatbot with Memory

Build a multi-turn chatbot that remembers context across messages

LangChain Memory ChatHistory Multi-Turn Session Management

■ Learning Objectives

✓ Understand why stateless LLMs need external memory management

✓ Implement ConversationBufferMemory for short conversations

✓ Implement ConversationSummaryMemory for long conversation compression

✓ Build a full multi-turn insurance advisor chatbot

✓ Manage multiple user sessions with isolated memory contexts

4.1 · Why Memory Matters

LLMs are **stateless** — each API call is independent with no memory of previous turns. To build a chatbot that remembers context (*'as I mentioned earlier, I have a pre-existing condition'*), we must explicitly pass conversation history with every request. LangChain's memory modules handle this bookkeeping automatically.

Memory Type	Best For	Limitation
ConversationBufferMemory	Short sessions, < 10 turns	Token cost grows unboundedly
ConversationSummaryMemory	Long sessions, > 20 turns	Summary may lose specific details
ConversationBufferWindowMemory	Fixed last K turns only	Old context is forgotten
ConversationTokenBufferMemory	Token-budget constrained	Requires token counting

4.2 · Full Implementation

```
touch ~/insurance-ai-lab/module4_memory_chatbot.py
```

Step 1 · Imports and Setup

```
# module4_memory_chatbot.py from dotenv import load_dotenv import os from
langchain_openai import ChatOpenAI from langchain.memory import (
ConversationBufferMemory, ConversationSummaryMemory, ConversationBufferWindowMemory )
from langchain.chains import ConversationChain from langchain_core.prompts import
PromptTemplate load_dotenv() llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3,
api_key=os.getenv("OPENAI_API_KEY"))
```

Step 2 · Custom Insurance Advisor Prompt

```
INSURANCE_ADVISOR_TEMPLATE = """You are Arjun, a knowledgeable and friendly insurance
advisor for SureShield Insurance, an Indian insurance company. You help customers
understand their options, resolve queries about existing policies, assist with claims,
```

```
and guide them towards suitable coverage. You remember everything the customer has told you in this conversation and use that context to give personalised, relevant responses. Conversation so far: {history} Customer: {input} Arjun:""" advisor_prompt = PromptTemplate( input_variables=["history", "input"], template=INSURANCE_ADVISOR_TEMPLATE )
```

Step 3 · Three Memory Implementations

```
# ■■ Option A: Buffer Memory (stores every message) ■■■■■■■■■■■■■■■■■■■■■■■■■ def create_buffer_chatbot(): memory = ConversationBufferMemory( memory_key="history", # matches template variable human_prefix="Customer", ai_prefix="Arjun", return_messages=False # returns string format ) chain = ConversationChain( llm=llm, memory=memory, prompt=advisor_prompt, verbose=False ) return chain, memory # ■■ Option B: Summary Memory (compresses older context) ■■■■■■■■■■■■■■■■■■■ def create_summary_chatbot(): memory = ConversationSummaryMemory( llm=llm, # uses LLM to generate summaries memory_key="history", human_prefix="Customer", ai_prefix="Arjun", return_messages=False ) chain = ConversationChain( llm=llm, memory=memory, prompt=advisor_prompt, verbose=False ) return chain, memory # ■■ Option C: Window Memory (only last K turns) ■■■■■■■■■■■■■■■■■■■■■■■■■■■ def create_window_chatbot(k: int = 5): memory = ConversationBufferWindowMemory( k=k, # keep last 5 exchanges memory_key="history", human_prefix="Customer", ai_prefix="Arjun", return_messages=False ) chain = ConversationChain( llm=llm, memory=memory, prompt=advisor_prompt, verbose=False ) return chain, memory
```

Step 4 · Multi-Session Manager

```
class InsuranceChatbotSessionManager: """Manages multiple isolated customer chat sessions.""" def __init__(self): self.sessions: dict = {} def get_or_create_session(self, session_id: str): """Get existing session or create a new one.""" if session_id not in self.sessions: chain, memory = create_buffer_chatbot() self.sessions[session_id] = { "chain": chain, "memory": memory, "turn_count": 0 } print(f"[Session {session_id}] New session created.") return self.sessions[session_id] def chat(self, session_id: str, user_message: str) -> str: """Send a message in a specific session and get a response.""" session = self.get_or_create_session(session_id) session["turn_count"] += 1 response = session["chain"].predict(input=user_message) return response def get_history(self, session_id: str) -> str: """Retrieve the conversation history for a session.""" if session_id not in self.sessions: return "Session not found." memory = self.sessions[session_id]["memory"] return memory.load_memory_variables({}).get("history", "No history yet.") def clear_session(self, session_id: str): """Clear memory for a specific session.""" if session_id in self.sessions: self.sessions[session_id]["memory"].clear() print(f"[Session {session_id}] Memory cleared.") def list_sessions(self): """List all active sessions with turn counts.""" for sid, data in self.sessions.items(): print(f" Session {sid}: {data['turn_count']} turns")
```

Step 5 · Test Multi-Turn Conversations

```
def demo_multi_turn_memory(): """Demonstrates memory retention across conversation turns.""" print("\n" + "="*65) print("DEMO: Multi-Turn Insurance Advisor Chatbot") print("="*65) manager = InsuranceChatbotSessionManager() session_id = "CUST-7001" # Conversation demonstrating memory retention conversation = [ "Hi, I'm Deepa. I have a family of 4 including two children aged 6 and 10.", "We're looking for health insurance. What would you recommend for us?", "What is the typical premium for a family floater plan covering Rs.5 lakhs?", "You mentioned a family floater earlier - does it cover my children's dental?", "My elder child has asthma. How does that affect the policy?", "Going back to what you said about pre-existing conditions - what's the waiting period?", "Can you summarise what you've recommended so far for my family?", ] for i, message in enumerate(conversation, 1): print(f"\n[Turn {i}] Customer: {message}")
```

```
response = manager.chat(session_id, message) print(f" Arjun: {response[:200]}...") if i
== 4: print("\n[Note: Agent correctly references 'family floater' from turn 2]")
print("\n\n--- FULL CONVERSATION HISTORY ---") print(manager.get_history(session_id)) #
Demonstrate session isolation print("\n\n--- TESTING SESSION ISOLATION ---") session2 =
"CUST-7002" print("\nNew Customer (different session):") resp = manager.chat(session2,
"Hello, what do you know about me from previous conversations?") print(f"Arjun: {resp}")
print("\n\nActive Sessions:") manager.list_sessions() def demo_summary_memory():
"""Shows how summary memory compresses long conversations."""  print("\n\n" + "="*65)
print("DEMO: Summary Memory (Long Conversation Compression)") print("="*65) chain,
memory = create_summary_chatbot() messages = [ "My name is Vikram and I'm 42 years
old.", "I'm a self-employed architect earning about Rs.2 lakh per month.", "I have an
existing term life policy of Rs.1 crore.", "I need additional coverage for critical
illness. What are my options?", "What riders can I add to my current term policy?", "How
does the claim process work for critical illness?", ] for msg in messages: response =
chain.predict(input=msg) print("\nFull Buffer would be very long.") print("\nSummary
Memory stores this compressed version:")
print(memory.load_memory_variables({})["history"]) if __name__ == "__main__":
demo_multi_turn_memory() demo_summary_memory()
```

4.3 · Running the Chatbot

```
python3 module4_memory_chatbot.py
```

■ ***Note:*** *Pay attention to Turn 4 and Turn 6 — the chatbot references specific details from earlier turns. This is the memory system working. Compare how the summary memory version produces a compressed but coherent history.*

Module 5 · Mem0 + Vector DB – Personalised Sessions

Store user preferences permanently and personalise responses across sessions

Mem0 Qdrant Vector DB Long-Term Memory Personalisation

■ Learning Objectives

✓ Understand the difference between session memory (Module 4) and persistent memory

✓ Set up Qdrant as a local vector database for semantic memory storage

✓ Integrate Mem0 to automatically extract and store user preferences

✓ Build an insurance agent that remembers user profiles across different sessions

✓ Retrieve and apply preferences like policy type, risk tolerance, and family details

5.1 · Persistent Memory Architecture

Module 4's memory is **ephemeral** — it lives only within one Python process run. When the program restarts, all context is lost. **Mem0** solves this by storing memories as semantic vectors in a persistent **Qdrant** vector database. When a user returns days later, the agent retrieves their stored preferences and picks up the conversation as if it never ended.

Component	Role
Mem0 Client	Extracts facts from conversations, manages memory lifecycle
Qdrant (local)	Vector database that stores embeddings of user memories
OpenAI Embeddings	Converts text memories to semantic vectors for similarity search
LangChain Agent	Uses retrieved memories to personalise every response

5.2 · Setup: Install and Start Qdrant

Qdrant can run in-process (no server needed) using the qdrant-client library with local storage. This is ideal for lab environments.

```
# Qdrant runs locally with file-based persistence — no Docker needed for labs pip
install qdrant-client mem0ai # Verify python3 -c "import qdrant_client, mem0;
print('Qdrant and Mem0 ready')"
```

■ ***Note:*** *For production, Qdrant would run as a Docker container or managed service. For this lab we use the embedded local mode which stores data in a ./qdrant_storage folder.*

5.3 · Full Implementation

```
touch ~/insurance-ai-lab/module5_persistent_memory.py
```

Step 1 · Imports and Mem0 Configuration

```
# module5_persistent_memory.py from dotenv import load_dotenv import os, json from mem0 import Memory from langchain_openai import ChatOpenAI from langchain_core.messages import SystemMessage, HumanMessage, AIMessage load_dotenv() # ■■ Mem0 Configuration with local Qdrant ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ MEM0_CONFIG = { "vector_store": { "provider": "qdrant", "config": { "collection_name": "insurance_user_memories", "path": "./qdrant_storage", # local persistent storage "on_disk": True, } }, "embedder": { "provider": "openai", "config": { "model": "text-embedding-3-small", "api_key": os.getenv("OPENAI_API_KEY") } }, "llm": { "provider": "openai", "config": { "model": "gpt-4o-mini", "temperature": 0, "api_key": os.getenv("OPENAI_API_KEY") } }, "version": "v1.1" } # Initialise Mem0 — this creates/loads the Qdrant collection memory_client = Memory.from_config(MEM0_CONFIG) # LangChain LLM for the agent llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3, api_key=os.getenv("OPENAI_API_KEY")) print("✓ Mem0 and Qdrant initialised successfully") print("✓ Memory storage: ./qdrant_storage")
```

Step 2 · Memory Management Helper Functions

```
def store_user_memory(user_id: str, content: str) -> None: """Extract and store facts from user message into Mem0.""" memory_client.add( messages=[{"role": "user", "content": content}], user_id=user_id, metadata={"source": "conversation", "domain": "insurance"} ) def retrieve_relevant_memories(user_id: str, query: str) -> str: """Retrieve memories relevant to the current query.""" memories = memory_client.search( query=query, user_id=user_id, limit=8 # retrieve top-8 most relevant memories ) if not memories.get("results"): return "No previous preferences stored for this user." memory_lines = [] for m in memories["results"]: memory_lines.append(f" - {m['memory']}") return "User's stored preferences and profile:\n" + "\n".join(memory_lines) def get_all_memories(user_id: str) -> list: """Get all stored memories for a user.""" result = memory_client.get_all(user_id=user_id) return result.get("results", []) def clear_user_memories(user_id: str) -> None: """Delete all memories for a user (for testing/reset).""" memory_client.delete_all(user_id=user_id) print(f"All memories cleared for user {user_id}")
```

Step 3 · Personalised Insurance Agent

```
class PersonalisedInsuranceAgent: """ An insurance advisor that remembers user preferences across sessions. Each conversation enriches the user's persistent memory profile. """ def __init__(self, user_id: str): self.user_id = user_id self.session_history = [] print(f"\n[Agent] Loaded session for user: {user_id}") # Check if returning user existing = get_all_memories(user_id) if existing: print(f"[Agent] Found {len(existing)} stored memories for this user") else: print("[Agent] New user — no prior memories found") def chat(self, user_message: str) -> str: """Process a message with memory-augmented context.""" # 1. Retrieve relevant memories for this query memories = retrieve_relevant_memories(self.user_id, user_message) # 2. Build context-aware system prompt system_prompt = f"""You are Arjun, an expert insurance advisor at SureShield Insurance. IMPORTANT - User's Stored Profile & Preferences: {memories} Use this profile to personalise every response. Reference specific details the user has shared previously (e.g., "As you mentioned, you have two children..." or "Given your preference for low-risk products..."). If the user shares new information about themselves, acknowledge it naturally. Current date: refer to general present context. Always be warm, professional, and genuinely helpful.""" # 3. Build message list with session history messages = [SystemMessage(content=system_prompt)] for turn in self.session_history[-6:]: # last 6 turns for context if turn["role"] == "human": messages.append(HumanMessage(content=turn["content"])) else: messages.append(AIMessage(content=turn["content"])) messages.append(HumanMessage(content=user_message)) # 4. Get LLM response response = llm.invoke(messages) ai_response = response.content # 5. Store new information from this exchange into Mem0 store_user_memory(self.user_id, f"User said: {user_message}") # 6.
```

```
Update session history self.session_history.append({"role": "human", "content": user_message}) self.session_history.append({"role": "ai", "content": ai_response}) return ai_response def show_memory_profile(self): """Display all stored memories for this user.""" memories = get_all_memories(self.user_id) print(f"\n[Memory Profile for {self.user_id}]") print(f"Total memories stored: {len(memories)}") for i, m in enumerate(memories, 1): print(f" {i}. {m.get('memory', 'N/A')}")
```

Step 4 · Simulate Two Separate Sessions

```
def simulate_session_1(user_id: str): """First session — user shares profile details.""" print("\n" + "="*65) print(f"SESSION 1 — First visit by {user_id}") print("="*65) agent = PersonalisedInsuranceAgent(user_id) conversations = [ "Hi, I'm Ananya Reddy. I'm 34 years old and married with one child (age 3).", "We live in Hyderabad. My husband works in IT and I'm a doctor.", "I'm very risk-averse and prefer policies with guaranteed returns.", "We're looking for health insurance with maternity and child coverage.", "Our combined monthly income is about Rs.3 lakhs.", "What health plan would you suggest for our family?", ] for msg in conversations: print(f"\nAnanya: {msg}") response = agent.chat(msg) print(f"Arjun: {response[:250]}...") print("\n[End of Session 1]") agent.show_memory_profile() def simulate_session_2(user_id: str): """Second session (new instance) — agent remembers everything.""" print("\n\n" + "="*65) print(f"SESSION 2 — Ananya returns after 3 days (NEW Python instance)") print("="*65) print("[New agent object created — no session history carried over]") print("[Only Mem0 persistent storage bridges the sessions]\n") agent = PersonalisedInsuranceAgent(user_id) # fresh instance new_conversations = [ "Hello, I'm back! Do you remember me?", "I wanted to follow up on the health insurance we discussed.", "We've decided to go with a higher sum insured — maybe Rs.10 lakhs family floater.", "Also, my husband mentioned we should add a term life policy too.", "Given everything you know about us, what's your recommendation?", ] for msg in new_conversations: print(f"\nAnanya: {msg}") response = agent.chat(msg) print(f"Arjun: {response[:300]}...") print() print("\n[End of Session 2]") agent.show_memory_profile() if __name__ == "__main__": USER_ID = "ananya_reddy_hyd_001" # Clear previous test data (optional - comment out to test persistence) clear_user_memories(USER_ID) # Run Session 1 — builds memory profile simulate_session_1(USER_ID) print("\n\n>>> Simulating 3-day gap — running Session 2 now <<<") print(">>> Note: agent is a completely new instance with no session history <<<") # Run Session 2 — retrieves from Mem0 simulate_session_2(USER_ID) print("\n\n" + "="*65) print("KEY OBSERVATION:") print("In Session 2, Arjun remembered:") print(" • Ananya's name, age, family details") print(" • Risk-averse profile preference") print(" • Hyderabad location, dual-income household") print(" • Maternity & child coverage requirements") print(" • Income bracket (Rs.3 lakhs/month)") print("This persistence comes from Mem0 + Qdrant, not session variables.") print("="*65)
```

5.4 · Running the Persistent Memory Agent

```
python3 module5_persistent_memory.py # After Session 1, check the qdrant_storage folder was created: ls -la ./qdrant_storage/ # Run again WITHOUT clear_user_memories to see true persistence: # Comment out: clear_user_memories(USER_ID) python3 module5_persistent_memory.py
```

■ **Important: The qdrant_storage folder persists between runs. To reset and start fresh, either call clear_user_memories() or delete the folder: rm -rf ./qdrant_storage**

Appendix A · Troubleshooting Guide

■ AuthenticationError: Incorrect API key

```
Check your .env file. Ensure the key starts with 'sk-'. Run: cat .env
Re-export: export OPENAI_API_KEY=sk-...
```

■ ModuleNotFoundError: No module named 'langchain'

```
Your virtual environment may not be active.
Run: source ~/insurance-ai-lab/venv/bin/activate
```

■ RateLimitError from OpenAI

```
You've hit the per-minute API rate limit.
Add time.sleep(2) between test calls in __main__.
```

■ Mem0 / Qdrant import errors

```
Reinstall: pip install mem0ai qdrant-client --upgrade
Check Python version is 3.10+: python3 --version
```

■ Agent loops endlessly (max_iterations hit)

```
Increase max_iterations or check tool return format.
Ensure tool return is a str, not a dict.
```

■ ValueError: Missing variables in prompt

```
Check ChatPromptTemplate variable names match agent_executor.invoke() keys.
Common fix: add agent_scratchpad to MessagesPlaceholder.
```

■ Qdrant collection error on second run

```
The collection already exists. This is fine — Mem0 handles this automatically.
If corrupt: rm -rf ./qdrant_storage and restart.
```

■ Code-Server terminal loses virtualenv

```
Run: source ~/insurance-ai-lab/venv/bin/activate
Add this to ~/.bashrc to auto-activate.
```

Appendix B · Full Requirements & Package Versions

Save the following as **requirements.txt** in your project directory and run **pip install -r requirements.txt** to ensure reproducibility.

```
# requirements.txt — LangChain + Mem0 Insurance AI Bootcamp # Core LangChain
langchain>=0.3.0 langchain-openai>=0.2.0 langchain-community>=0.3.0
langchain-core>=0.3.0 # OpenAI SDK openai>=1.40.0 # Vector Database qdrant-client>=1.9.0
# Persistent Memory mem0ai>=0.1.0 # Utilities python-dotenv>=1.0.0 requests>=2.31.0
colorama>=0.4.6 # Optional: for advanced tools pydantic>=2.0.0
```

Quick Install Command

```
cd ~/insurance-ai-lab source venv/bin/activate pip install -r requirements.txt
```

Module-to-File Reference

Module	File	Key Library
1 · Claim Validation	module1_claim_agent.py	langchain.agents, create_tool_calling_agent
2 · Tool Calling	module2_tool_calling.py	langchain_core.tools, @tool decorator
3 · ReAct Agent	module3_react_agent.py	AgentExecutor, ReAct prompt pattern
4 · Memory Chatbot	module4_memory_chatbot.py	ConversationBufferMemory, ConversationChain
5 · Mem0 Persistence	module5_persistent_memory.py	mem0.Memory, qdrant-client